ZHUOER DU

346-513-6663 | zhuoerdu@umich.edu | linkedin.com/in/wendy-du | github.com/w-dze

EDUCATION

University of Michigan, College of Engineering | Ann Arbor, MI

Bachelor of Science in Engineering in Computer Science | Minor: Statistics | Expected May 2028

- GPA: 3.74
- **Honors**: UM Club of Houston Scholarship, Dean's Honor List, University Honors
- **Relevant Coursework**: Data Structures & Algorithms, Computer Networks, Computer Security, Web Systems, Computer Organization, Machine Learning, Database Management Systems

INTERNSHIP EXPERIENCE

Daikin Comfort Technologies North America | Waller, TX

Software Engineer Intern | May 2026 – July 2026

- Designed a TypeScript and Playwright automated testing framework validating **RESTful API** endpoints across thermostat models and 3 firmware releases.
- Cut validation time from 20 hours down to just 17 minutes through the automated testing framework, freeing the QA team for exploratory investigation.
- Built Python monitoring and load-testing tools to troubleshoot local API reliability under sustained thermostat traffic during peak usage periods.
- Identified a 500ms minimum stable inter-request delay through the load-testing tools, accelerating defect diagnosis for the whole firmware team.
- Developed a reusable TypeScript device-discovery module with polling and retry logic, syncing local and cloud device state via **CI/CD**-gated releases.

PROJECTS

Relational Database Engine | C++

- Built a CLI relational database engine in C++ supporting CREATE, INSERT, DELETE, PRINT, and JOIN operations across multiple linked tables.
- Applied object-oriented design to manage tables, records, and query execution, keeping the engine's core modules cleanly separated by responsibility.
- Sped up record lookup and filtering across relational datasets by implementing hash-table and binary-search-tree indexes for the query planner used.
- Handled invalid commands, edge cases, and multiple query paths to strengthen correctness, applying core data structures and debugging techniques.

Syllabus Visualizer | TypeScript, Next.js, React, PostgreSQL, Prisma, Gemini API

- Created a full-stack Next.js application transforming unstructured syllabi into interactive course schedules for students managing multiple classes.
- Cut a recurring student workflow from 20 minutes down to just 2 minutes by automating the entire syllabus-to-schedule conversion process end to end.
- Designed backend API and **PostgreSQL**/Prisma workflows to extract, validate, persist, and serve structured course data to a responsive React interface.
- Deployed production app to Vercel with **Dockerized** local development environment, using **Gemini API** schema validation to prevent malformed outputs.

LCDC Infrastructure Pipeline | Python, React, Regular Expressions, GIS

- Developed a Python data-processing pipeline transforming public infrastructure records into validated, GIS-ready data for downstream mapping analysis.
- Cut manual processing time by roughly 80% through the automated pipeline, supporting adoption across three separate local government office teams.
- Implemented a React interface with reusable parsing and validation workflows, automating processing of heterogeneous public infrastructure records.
- Worked with developers and government stakeholders through **Agile** sprints, code reviews, and iterative feedback to ship maintainable software for users.

Real-Time Notification Service | Node.js, Kafka, Redis, Docker, Kubernetes, AWS EC2

- Built a real-time notification service in Node.js, streaming user activity events through **Kafka** to downstream subscriber services reliably each time.
- Cached frequently accessed user preferences in **Redis**, cutting notification delivery latency significantly during peak traffic windows tested overall.
- Containerized the service with **Docker** and deployed it on **Kubernetes**, configuring horizontal pod autoscaling for bursty notification traffic volume.
- Hosted the deployment on **AWS EC2** behind an Application Load Balancer, routing traffic across replicas for consistent availability under heavy load.
- Load-tested the pipeline with simulated concurrent users, tuning Kafka partition counts and Redis eviction policies to hold latency perfectly steady.

TECHNICAL SKILLS

Languages: TypeScript/JavaScript, Python, C++, C, Java, SQL, HTML/CSS

Frameworks & Libraries: React, Next.js, Node.js, Flask, Prisma

Databases & APIs: PostgreSQL, SQLite, REST APIs

Tools & Infrastructure: Git/GitHub, Docker, AWS EC2, Vercel, Playwright, Linux/Unix